MVLU COLLEGE
AI PRACTICAL NO. 7

AIM:Support Vector Machines (SVM)

- Implement the SVM algorithm for binary classification.
- Train an SVM model using a given dataset and optimize its parameters.
- Evaluate the performance of the SVM model on test data and analyze the results.

#CODE:

```
# PRACTICAL 7: SUPPORT VECTOR MACHINE (SVM)
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
# Upload dataset
uploaded = files.upload()
print("First 5 Rows:")
print(df.head())
print("\nDataset Shape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nClass Distribution:")
print(df["Outcome"].value_counts())
# Features and target
X = df.drop(columns="Outcome")
y = df["Outcome"]
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)
# SVM + Parameter Optimization
pipe = Pipeline([
    ("scaler", StandardScaler()),
```

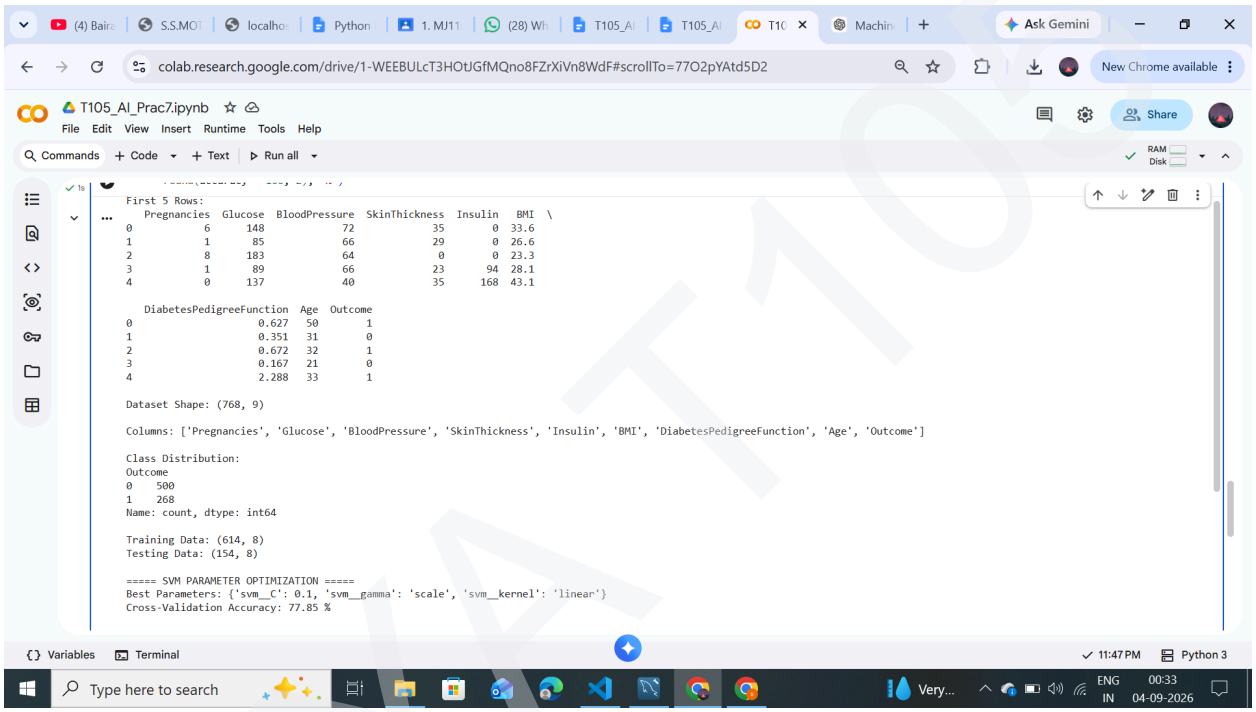
```
    ("svm", SVC())
])
params = {
    "svm__C": [0.1, 1, 10],
    "svm__kernel": ["linear", "rbf"],
    "svm__gamma": ["scale", "auto"]
}
grid = GridSearchCV(pipe, params, cv=5, scoring="accuracy")
grid.fit(X_train, y_train)
print("\n===== SVM PARAMETER OPTIMIZATION =====")
print("Best Parameters:", grid.best_params_)
print("Cross-Validation Accuracy:",
      round(grid.best_score_ * 100, 2), "%")
# Best model prediction
model = grid.best_estimator_
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("\n===== SVM TEST PERFORMANCE =====")
print("Test Accuracy:", round(accuracy * 100, 2), "%")
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
plt.figure(figsize=(6, 5))
plt.imshow(cm)
plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.xticks([0, 1])
plt.yticks([0, 1])
for i in range(2):
    for j in range(2):
        plt.text(j, i, cm[i, j], ha="center", va="center")
plt.colorbar()
plt.show()
# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# Actual vs Predicted
result = pd.DataFrame({
```

```
    "Actual Outcome": y_test.values,
    "Predicted Outcome": y_pred
})
print("\n===== SVM ACTUAL VS PREDICTED =====")
print(result.head(15))
# Actual vs Predicted Graph
plt.figure(figsize=(8, 5))
plt.plot(y_test.values, marker="o", label="Actual")
plt.plot(y_pred, marker="x", label="Predicted")
plt.xlabel("Test Sample")
plt.ylabel("Class")
plt.title("SVM Actual vs Predicted")
plt.legend()
plt.grid()
plt.show()
# Kernel Comparison
kernels = ["linear", "rbf"]
kernel_acc = []
for k in kernels:
    m = Pipeline([
        ("scaler", StandardScaler()),
        ("svm", SVC(
            C=grid.best_params_["svm__C"],
            kernel=k,
            gamma=grid.best_params_["svm__gamma"]
        ))
    ])
    m.fit(X_train, y_train)
    kernel_acc.append(
      accuracy_score(y_test, m.predict(X_test))
    )
# Kernel Graph
plt.figure(figsize=(7, 5))
plt.bar(kernels, kernel_acc)
plt.xlabel("SVM Kernel")
plt.ylabel("Accuracy")
plt.title("SVM Kernel Comparison")
plt.ylim(0, 1)
plt.show()
# Final Result
```

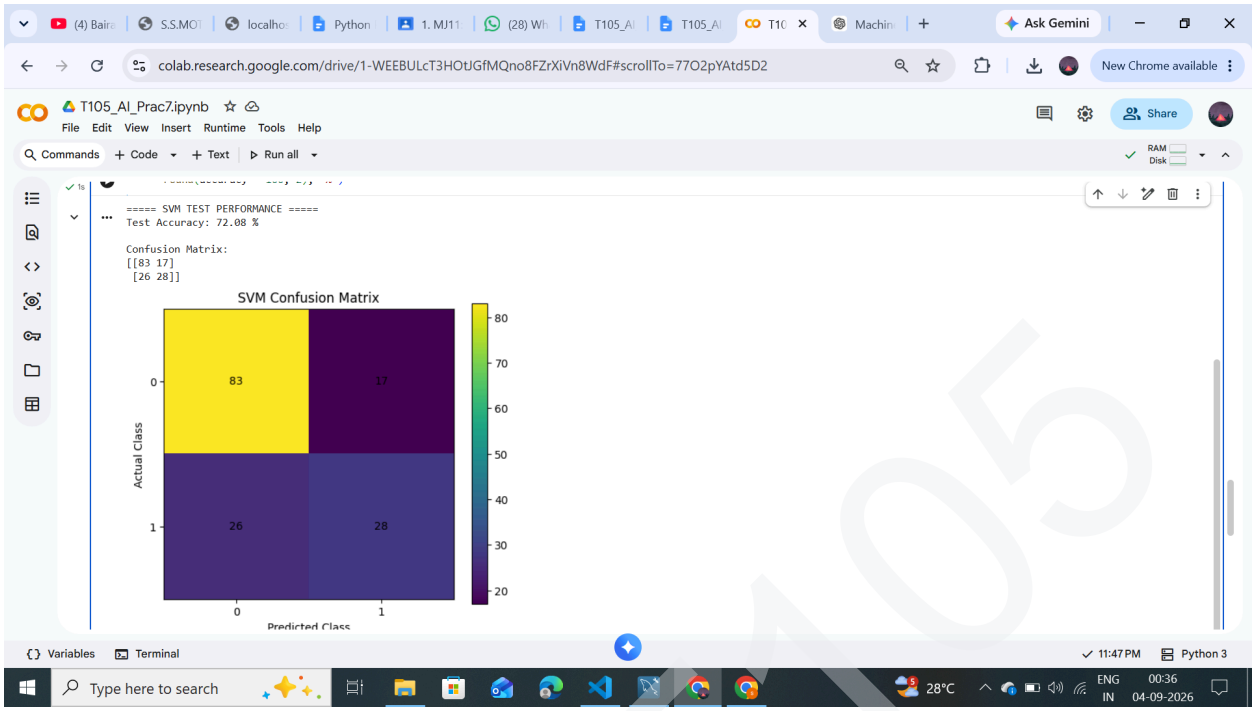
MVLU COLLEGE
AI PRACTICAL NO. 7

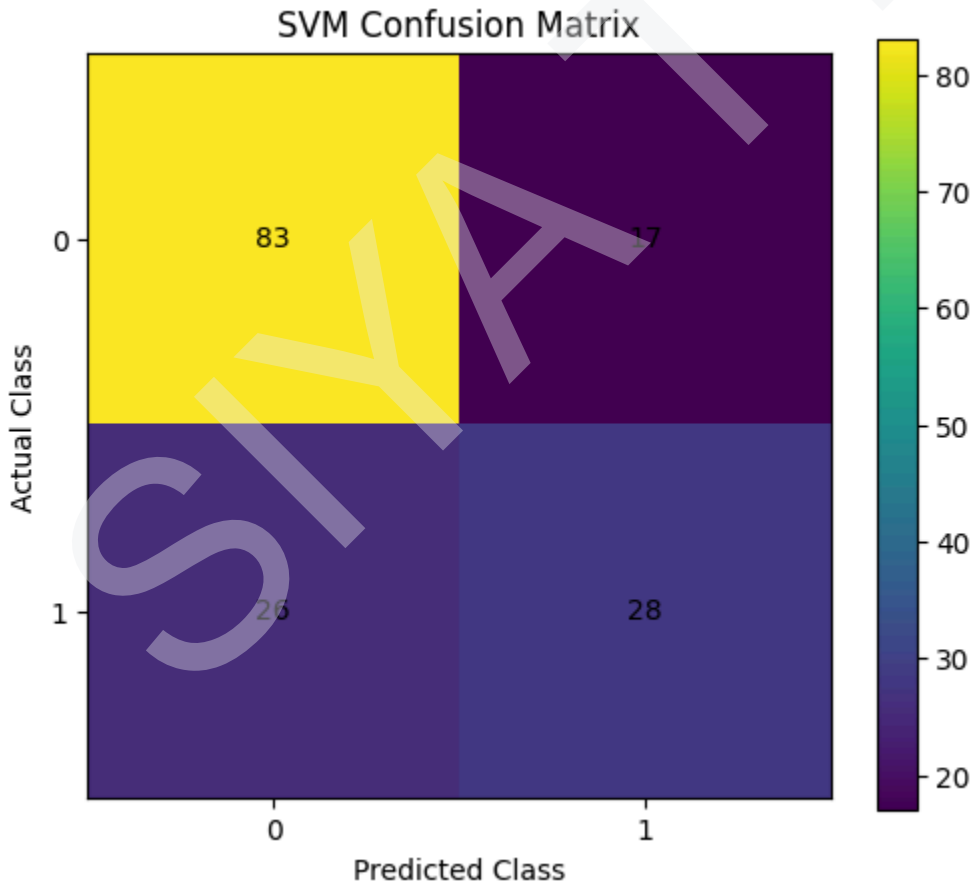
```
print("\n===== FINAL RESULT =====")
print("Best Parameters:", grid.best_params_)
print("Cross-Validation Accuracy:",
      round(grid.best_score_ * 100, 2), "%")
print("Test Accuracy:",
      round(accuracy * 100, 2), "%")
```

#OUTPUT:

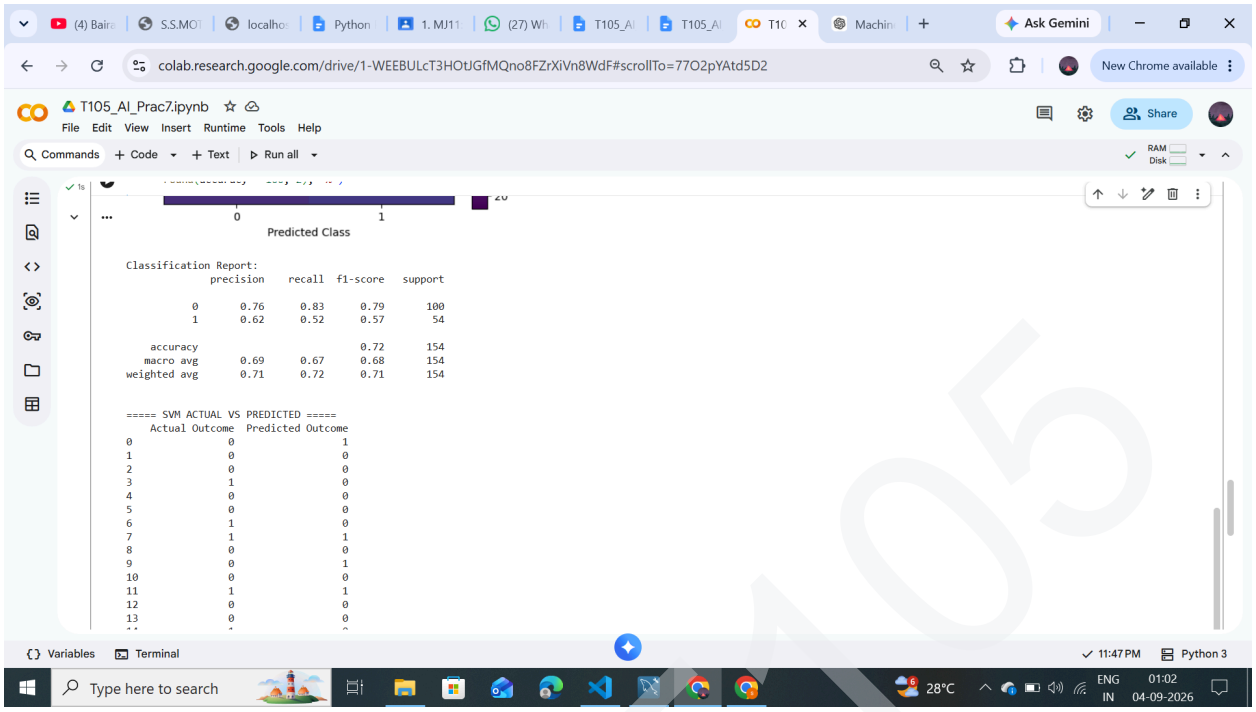


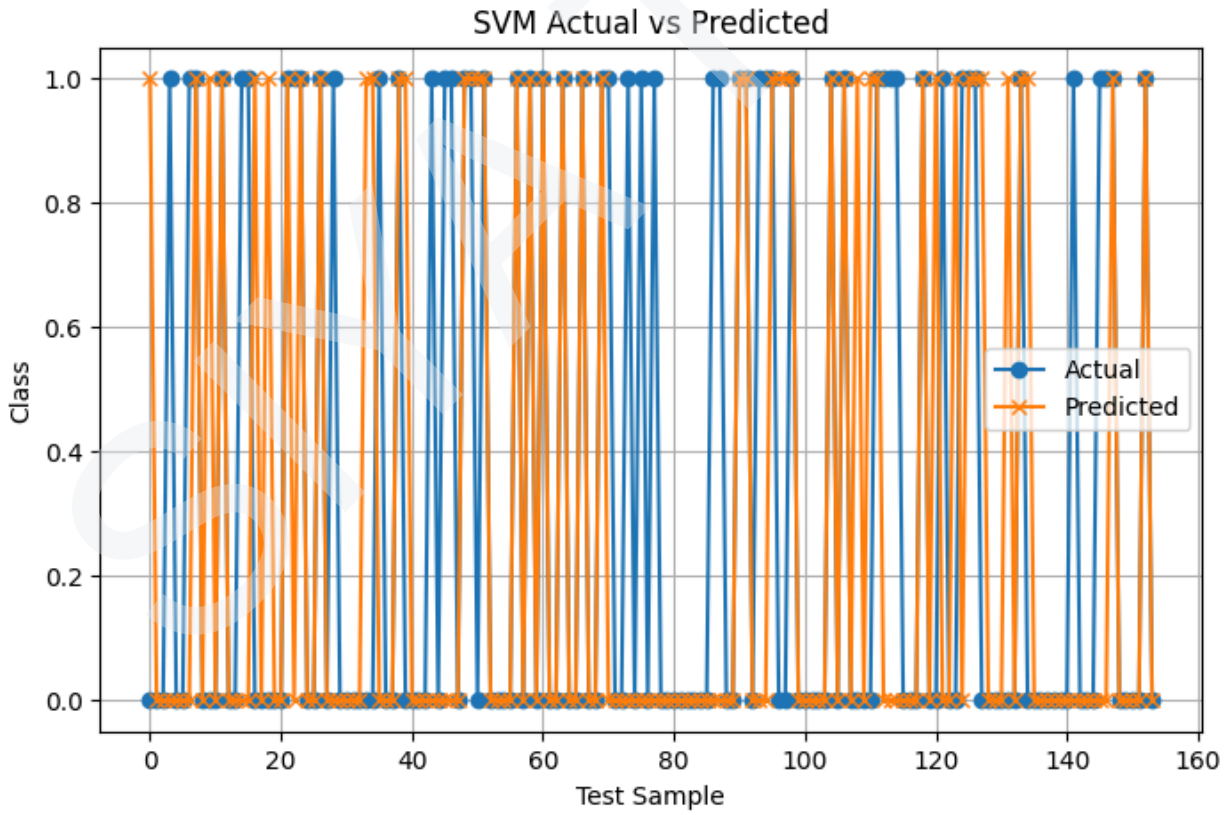
MVLU COLLEGE
AI PRACTICAL NO. 7





MVLU COLLEGE
AI PRACTICAL NO. 7





MVLU COLLEGE
AI PRACTICAL NO. 7